



Smart Transport Operations Platform

Post-Phase-1 Development Report

Phases 2 – 5 · RBAC Enforcement · UI/UX Upgrades

Prepared for

Date

Version

Odoo Virtual Hackathon

July 2026

1.0

Phases 2 · 3 · 4 · 5 completed · Full RBAC · Plotly Analytics · PDF Report

Executive Summary

TransitOps is a **Smart Transport Operations Platform** built for the Odoo Virtual Hackathon. It provides a complete fleet management solution covering vehicle tracking, driver management, trip dispatch, maintenance workflows, fuel/expense logging, and analytics — all secured behind a two-layer Role-Based Access Control (RBAC) system.

Phase	Scope	Status
Phase 1	Auth · DB Models · Seed · Dashboard · Dark Theme · Router	■ Complete
Phase 2	Vehicle Registry CRUD · Driver Management CRUD · Business Rules	■ Complete
Phase 3	Trip Dispatch Engine · Lifecycle · Validation	■ Complete
Phase 4	Maintenance Workflow · Fuel & Expense Logging	■ Complete
Phase 5	Analytics (5 charts) · CSV Export · Settings · User Mgmt	■ Complete
RBAC	Two-layer permission system: Sidebar + Per-button enforcement	■ Complete
UI/UX	Selection cards · CSS polish · Dashboard overhaul	■ Complete

Key Numbers at a Glance

8	8	5	5	Pydantic Schem
4	44	5	8	Bugs Fixed

Phase 2

Vehicle & Driver Management

Full CRUD · schemas · services · RBAC · business rules

Phase 2 delivers complete Create-Read-Update-Delete operations for the two core fleet entities — **Vehicles** and **Drivers** — following a strict three-layer architecture: *Pydantic schema* → *Service* → *Streamlit page*.

2.1 New Files

File	Layer	Purpose
app/schemas/vehicle_schema.py	Schema	VehicleCreate, VehicleUpdate — field validation, type coercion
app/schemas/driver_schema.py	Schema	DriverCreate, DriverUpdate — license, score, expiry validation
app/services/vehicle_service.py	Service	get_all, get_by_id, create, update, delete, fleet_summary
app/services/driver_service.py	Service	get_all, get_by_id, create, update, delete, driver_summary
frontend/pages/page_fleet.py	UI	5-KPI bar, type/status filters, add form, edit form, status change, soft delete
frontend/pages/page_drivers.py	UI	6-KPI bar, license expiry alerts, add/edit forms, status change, soft delete

2.2 Business Rules

- Registration number must be unique — service raises **ValueError** on duplicate.
- Safety score < 70 → driver is **auto-suspended** on create or update.
- Vehicles with trip history are **Retired** (not hard-deleted); drivers set to **Off Duty**.
- Driver license expiry < today → flagged **EXPIRED** with a red banner across the page.
- License expiry ≤ 30 days → **EXPIRING SOON** amber warning banner.
- Cannot manually set a vehicle to Available while a Dispatched trip uses it.

2.3 UI Features

Feature	Details
5-KPI header (Fleet)	Total · Available · On Trip · In Maintenance · Retired
6-KPI header (Drivers)	Total · Available · On Trip · Suspended · Expired Licence · Expiring ≤30d

Feature	Details
Filter bar — Fleet	Type dropdown + Status dropdown
Filter bar — Drivers	Status + Licence Category + Name search (text input)
Colour-coded table	Status column styled green / amber / red / gray via Pandas Styler
Dynamic tabs	Add tab appears only when role has can("fleet.add") permission
Selection card	Orange left-border card with model, type, capacity, odometer, cost, status badge
Actions row	Edit + Delete buttons; Status change in its own 3:1 row below card

Phase 3

Trip Dispatch Engine

Full lifecycle · business rule validation · status automation

Phase 3 implements the core operational workflow: create a trip, validate all business constraints, **dispatch** it (locking vehicle + driver), **complete** it (releasing resources + recording fuel/odometer), or **cancel** it at any actionable stage.

3.1 Trip Lifecycle

Status	Trigger	Vehicle Status	Driver Status
Draft	Trip created	Available (unchanged)	Available (unchanged)
Dispatched	Dispatch action	On Trip	On Trip
Completed	Complete action	Available	Available
Cancelled	Cancel action	Available *	Available *

* Released only if the trip was *Dispatched* at time of cancellation.

3.2 Dispatch Validation Rules

- Vehicle status must be **Available** at dispatch time.
- Driver status must be **Available** at dispatch time.
- Driver licence must **not** be expired — hard block (shown as red error, not a warning).
- **cargo_weight_kg** must be $\leq$ **vehicle.max_capacity_kg**.
- **planned_distance_km** must be > 0 ; revenue and cargo weight must be ≥ 0 .
- Trip code is auto-generated: **TRP-DDMMYY-XXXXXX** (date + 6-char UUID fragment).
- Soft capacity check also runs at *trip creation* (Draft) to catch obvious errors early.

3.3 New Files

File	Purpose
app/schemas/trip_schema.py	TripCreate · TripUpdate · TripComplete with Pydantic v2 validators
app/services/trip_service.py	create · dispatch · complete · cancel · get_available_vehicles/drivers
frontend/pages/page_trips.py	6-KPI header · All Trips tab · Create Trip tab · Actions panel

■ ■ The Dispatcher role has full trip workflow access (create, dispatch, complete, cancel). Fleet Manager also has full access. Safety Officer and Financial Analyst are view-only.

Phase 4

Maintenance & Fuel / Expense Logging

Auto vehicle status transitions · cost tracking

4.1 Maintenance Workflow

Maintenance records act as **automatic state machines** for vehicle availability. Logging puts a vehicle In Shop; closing releases it back to Available — no manual status change needed by the user.

- Vehicle must **not** be On Trip before logging (cannot repair a moving vehicle).
- Vehicle must **not** already be In Shop — must close existing record first.
- Logging maintenance → vehicle status: **Available** → **In Shop** (automatic).
- Closing maintenance → vehicle status: **In Shop** → **Available** (automatic).
- Only one **Active** maintenance record allowed per vehicle at a time.
- Closing allows an optional **actual_cost** override (defaults to estimated cost).

4.2 Fuel & Expense Entities

Entity	Key Fields	Validation
FuelLog	vehicle · liters · cost · date · trip (optional)	liters > 0; cost > 0; trip must belong to same vehicle
Expense	vehicle · category · amount · date	category must be in defined list (7 categories)

Expense Categories: Toll · Insurance · Fine · Parking · Loading/Unloading · Driver Allowance · Misc

4.3 New Files

File	Purpose
app/schemas/maintenance_schema.py	MaintenanceCreate · MaintenanceClose
app/schemas/fuel_schema.py	FuelCreate · ExpenseCreate · EXPENSE_CATEGORIES list
app/services/maintenance_service.py	log · close · summary · vehicle selector helpers
app/services/fuel_service.py	add_fuel_log · add_expense · cost_summary · get_all
frontend/pages/page_maintenance.py	4-KPI header · Log tab · All Records · Close panel (FM only)
frontend/pages/page_fuel_expenses.py	Dynamic 4-tab: Log Fuel · Log Expense · Fuel Records · Expense Records

Phase 5

Analytics & Settings

5 Plotly charts · CSV export · user management · danger zone

5.1 Analytics Charts

Chart	Type	Metric	Colour Logic
Revenue by Vehicle	Bar	Trip.revenue (Completed only)	Green bars
Completed Trips by Month	Bar	Trip.created_at grouped monthly	Blue bars
Fuel Efficiency per Vehicle	Bar	planned_distance_km / fuel_consumed_l	Green ≥10 · Amber ≥7 · Red <7 km/L
Cost Breakdown	Donut	Fuel cost + Expense amount + Maint. cost	3-segment donut
Total Trips per Vehicle	Bar	All trips (any status) by vehicle	Purple bars

■ All charts use the app's dark theme palette via Plotly's `paper_bgcolor='rgba(0,0,0,0)'`. CSV export (Trips · Fuel Logs · Expenses) is visible only to Fleet Manager and Financial Analyst roles.

5.2 Settings Page Tabs

Tab	Access	Content
System Info	All roles	App name · DB type · framework version · ORM · debug mode
User Management	Fleet Manager	Role-coloured user table + role-change form
RBAC Matrix	Fleet Manager	Read-only colour-coded view of all 4 roles × 8 modules
Danger Zone	Fleet Manager	Reset Database button — deletes + reseeds data (dev tool)

RBAC — Two-Layer Permission System

Access control is enforced at **two independent layers**, ensuring that no action can be performed by an unauthorised role — even if they navigate directly to a URL.

• Maps: role → module → 'none' / 'view' / 'edit'

• Effect: Side

• Maps: role → action → True / False

• 44 named

Permission Matrix

Module	Fleet Manager	Dispatcher	Safety Officer	Financial Analyst
Dashboard	■ View	■ View	■ View	■ View
Fleet	—■ Full CRUD	■ View only	■ View only	■ View only
Drivers	—■ Full CRUD	■ View only	—■ Add / Edit / Status	■ View only
Trips	—■ Full workflow	—■ Full workflow	■ View only	■ View only
Maintenance	—■ Full CRUD	■ View only	■ View only	■ View only
Fuel & Expenses	—■ Full CRUD	■ View only	■ View only	—■ Add / View
Analytics	■ + CSV Export	■ Charts only	■ Charts only	■ + CSV Export
Settings	—■ Full access	■ Hidden	■ Hidden	■ Hidden

Code Pattern — Using can()

```
from frontend.components.auth_guard import can

# Show 'Add Vehicle' tab only if role permits
if can('fleet.add'):
    tabs.append('■ Add Vehicle')

# Show Dispatch button only for valid status + permission
if can('trips.dispatch') and trip['status'] == 'Draft':
    show_dispatch_button()
```

UI / UX Upgrades

Dashboard Overhaul

- **Removed dead filter row** — Type / Status / Region dropdowns had no effect and were confusing. Removed entirely.
- **Recent Trips — now live** — Added Vehicle and Driver columns; queries the live DB on every render. Previously showed only Trip Code, Route, Distance, Status.
- **Flat bar → Donut chart** — Fleet Status is now a donut with the total vehicle count as the centre label, plus a mini 3-stat row (Available · On Trip · In Shop) below.
- **8th KPI tile** — Total Revenue (Rs.) from completed trips added to the KPI bar.

Selection Card — Actions Panel Redesign

All three management pages (Fleet, Drivers, Trips) previously had a cramped three-column layout where the status dropdown was squeezed between the Edit and Delete buttons. This has been replaced with a **styled Selection Card** system.

Before	After
Dropdown → [Edit] [Status dropdown + Apply] [Delete] • Status change forced into the middle column • No item details visible • 4 fixed columns regardless of role	Dropdown → Orange left-border card (shows all key fields) • Then: [Edit] [Delete] buttons (dynamic count) • Then: — Change Status — wide 3:1 row • Only buttons valid for current status + role appear

CSS Enhancements (style.css)

Element	Enhancement
Primary Button	Gradient fill (orange→dark orange) · glow box-shadow · translateY(-1px) hover lift
Secondary Button	Border colour transitions to orange on hover
Form Inputs	Border highlights orange on focus with soft 2px glow ring
Dataframe Tables	border-radius: 10px on all st.dataframe() outputs
Scrollbar	Custom 6px dark scrollbar matching dark theme palette
.selection-card	New CSS class — orange left-border · flex-wrap meta tags · hover effect
.sc-badge-*	6 colour variants (green / yellow / red / gray / blue / purple)

Errors Encountered & Resolutions

This section documents every notable error or bug encountered during development, along with its root cause and the resolution applied.

# 1	StreamlitAPIException — st.switch_page() page not found	High
Root Cause	st.switch_page() requires files inside a root-level pages/ folder. Our architecture places pages in frontend/pages/ (per SDD) which Streamlit's native router doesn't support.	
Resolution	Replaced st.switch_page() with a custom PAGE_MAP router in frontend/router.py. Single app.py entry point; navigation is driven by session_state['current_page'].	
# 2	UnicodeEncodeError — cp1252 codec can't encode emoji	Low
Root Cause	Windows PowerShell terminal uses cp1252 encoding. Python's print() crashes on Unicode emoji used in log messages and test output.	
Resolution	Added sys.stdout.reconfigure(encoding='utf-8', errors='replace') at the top of all CLI scripts (smoke tests, seed, PDF generator).	
# 3	SQLite OperationalError — database is locked	Medium
Root Cause	Smoke tests attempted reset_database() (deletes .db file) while Streamlit held an open SQLAlchemy connection to the same file.	
Resolution	Always stop Streamlit before running tests. reset_database() now prints a clear error message. Documented in developer workflow guide.	
# 4	SyntaxError — walrus operator := in assert statement	Low
Root Cause	Smoke test used assert f.cost_per_l_calc := round(...) which is invalid Python syntax. The walrus operator cannot be used inside assert.	
Resolution	Removed the malformed assertion. cost_per_l is computed inside the service layer and returned in the response dict — no assertion needed.	

#5 Pydantic ValidationError — model_post_init not called for None date Medium

Root Cause MaintenanceCreate used date: datetime.date = None as a field default. Pydantic v2 does not call model_post_init for None defaults the same way v1 did.

Resolution Changed to use model_post_init with object.__setattr__ for the default date assignment — consistent with the pattern in FuelCreate and ExpenseCreate.

#6 Protobuf — Descriptors cannot be created directly Low

Root Cause Importing any Streamlit page module outside the Streamlit runtime raises a protobuf version conflict. Happens when smoke tests try to import frontend.pages.*

Resolution Backend-only smoke tests skip all frontend page imports. Pages are verified by running the full Streamlit app and manually exercising each role. Known limitation.

#7 Dashboard — Recent Trips showing no Vehicle or Driver Medium

Root Cause The DB query existed but the resulting dict only had Trip Code, Route, Distance, Status — no joins for vehicle or driver names. Filter dropdowns (Type / Status / Region) had no underlying logic and confused users.

Resolution Added v_map and d_map lookups inside _get_dashboard_data(). Recent Trips now shows 6 columns. Removed the dead filter row entirely.

#8 KeyError: 'can_edit_v' — list comprehension scope bug High

Root Cause The fleet actions panel used locals()[c] inside a list comprehension to check boolean flags. In Python 3, list comprehensions have their own scope, so locals() returns only the comprehension's variables, not the enclosing function's.

Resolution Removed the broken list comprehension. The button count was already tracked by num_btns = sum([can_edit_v, can_delete_v]) — used that directly.

Architecture Overview

Design Principles

- **Zero-Streamlit backend** — `app/` contains no streamlit imports. All business logic is pure Python. Only `frontend/pages/` calls `st.*`
- **Custom router** — `PAGE_MAP` in `router.py` replaces Streamlit's native multipage system, enabling full RBAC control and our folder structure.
- **SQLite + SQLAlchemy ORM** — No server needed for hackathon. Typed `Mapped[]` fields, `reset_database()` for development iterations.
- **Pydantic v2 schemas** — All user input validated before reaching services. `ValidationError` messages shown via `st.error()`. Services only receive clean data.
- **bcrypt authentication** — Passwords hashed at seed time. Sessions via `st.session_state` with `get_current_user()` helper.

Request Flow

Step	Action	Code Location
1	User logs in	<code>app.py</code> validates credentials via <code>auth_service.login()</code>
2	Sidebar renders	<code>sidebar.py</code> filters <code>SIDEBAR_ITEMS</code> by <code>has_access(role, module)</code>
3	Page selected	<code>session_state['current_page']</code> updated; <code>router.py</code> calls <code>page.render()</code>
4	Page loads	<code>require_auth()</code> verifies session; <code>can(action)</code> gates UI components
5	Form submitted	Pydantic schema validates → service processes → DB updated
6	Page re-renders	Streamlit re-runs; all data fetched fresh from DB

Directory Structure

```
TransitOps/
├── app.py Entry point · login · sidebar · router
├── .env DATABASE_URL · SECRET_KEY · APP_NAME · DEBUG
├── .streamlit/config.toml Dark theme · orange accent
├── requirements.txt
├──
├── app/ Backend (zero Streamlit imports)
│   ├── config.py Reads .env · typed settings constants
│   ├── constants.py Status enums: VehicleStatus, DriverStatus ...
│   └── logger.py get_logger(__name__) rotating file handler
```

```

■ ■■■ auth/
■ ■ ■■■ hashing.py bcrypt hash_password / verify_password
■ ■ ■■■ rbac.py PERMISSIONS + ROLE_ACTIONS + role_can() + can_edit()
■ ■■■ database/
■ ■ ■■■ engine.py get_session · init_db · reset_database
■ ■ ■■■ seed.py 4 roles · 4 users · 6 vehicles · 5 drivers · 5 trips
■ ■■■ models/ 8 SQLAlchemy ORM models
■ ■ ■■■ role · user · vehicle · driver · trip · maintenance · fuel_log · expense
■ ■■■ schemas/ Pydantic v2 validation schemas
■ ■ ■■■ vehicle · driver · trip · maintenance · fuel
■ ■■■ services/ Business logic – pure Python
■ ■■■ auth · vehicle · driver · trip · maintenance · fuel
■
■ ■■■ frontend/
■ ■■■ router.py PAGE_MAP + route_to_page() dispatcher
■ ■■■ assets/style.css Global dark theme CSS
■ ■■■ components/
■ ■ ■■■ auth_guard.py require_auth() · require_role() · can(action)
■ ■ ■■■ kpi_card.py render_kpi_card(value, label, color)
■ ■ ■■■ sidebar.py Role-filtered navigation sidebar
■ ■■■ pages/ Each file: render() only
■ ■■■ dashboard · fleet · drivers · trips · maintenance
■ fuel_expenses · analytics · settings
■
■ ■■■ docs/
■ ■■■ TransitOps_Phase1_Report.pdf
■ ■■■ TransitOps_Phases2to5_Report.pdf
■ ■■■ generate_report_p2_p5.py
■ ■■■ data/transitops.db Auto-created SQLite database

```